

AI4Everyone

Corso di Formazione su Claude

Manuale Utente — Seconda Lezione

Data sessione: 20 marzo 2026

Docente: Stefano Zaghi

Partecipanti: Partecipante 1, Partecipante 2, Partecipante 3, Partecipante 4

Piattaforma: Microsoft Teams | Durata: ~2 ore

Sommario

INTRODUZIONE.....	4
Contesto e Rilevanza	4
Obiettivi di Apprendimento	4
1 — RIEPILOGO DELLA PRIMA LEZIONE	5
1.1 Framework TCOF	5
1.2 Context Window	6
1.3 Gestione della Memoria.....	6
2 — FEEDBACK DEI PARTECIPANTI	7
2.1 Meta-Prompt come Regola nelle Preferenze	7
2.2 Script Python per Automazione Documentale.....	7
2.3 Artifact come Checkpoint di Contesto	8
2.4 Approccio Esplorativo e Incrementale.....	8
3 — PROGETTI CLAUDE.....	9
3.1 Cos'è un Progetto Claude.....	9
3.2 Quando Usare un Progetto.....	9
3.3 Struttura dell'Interfaccia Progetti.....	10
3.4 Creazione di un Progetto.....	10
3.5 Le Quattro Aree di un Progetto	11
3.5.1 Memory (Memoria Autogenerata).....	11
3.5.2 Instructions (Istruzioni di Progetto).....	11
3.5.3 Files (Project Knowledge Base)	12
3.5.4 Tool Access e Connettori	12
3.6 Gerarchia della Memoria in Claude	13
4 — WEB SEARCH VS DEEP RESEARCH	14
4.1 Web Search.....	14
4.2 Deep Research (Research).....	14
5 — EXTENDED THINKING E SELEZIONE DEL MODELLO.....	15
5.1 Extended Thinking.....	15
5.2 Modelli Disponibili.....	15
6 — PROJECT KNOWLEDGE.....	16
6.1 Come Funziona il Database Vettoriale.....	16
6.2 Estrapolazione dei Chunk.....	17
7 — META-PROMPTING: GENERAZIONE DEI PROMPT	18
7.1 Principio del Meta-Prompting.....	18
7.2 Applicazione alla Configurazione di Progetto.....	18
Variante A — Prompt Diretto.....	18
Variante B — Prompt per Generare il Prompt	19
7.3 Qualità Riproducibile con Prompt Ben Strutturati	19
8 — ESERCIZIO PRATICO: IL BEAS ASSISTANT	20
8.1 Problema: Accedere alla Documentazione Online di Beas	20

8.2 Step 1 — Creazione del Progetto	20
8.3 Step 2 — Generazione della Configurazione tramite Meta-Prompting	21
8.4 Step 3 — Configurazione del Progetto con i Deliverable	21
8.5 Note sulla Configurazione Generata	21
9 — COWORK: AUTOMAZIONE AGENTICA	22
9.1 Cos'è Cowork	22
9.2 Come si Avvia un Task in Cowork	22
9.3 Sotto-agenti e Parallelizzazione	23
9.4 Cowork per la Documentazione di Beas	23
9.5 Qualità dell'output in Cowork	24
10 — ESERCIZI ASSEGNATI	25
Esercizio 1 — Assistente Virtuale Personale	25
Esercizio 2 — Formatter di Prompt TCOF	25
Esercizio 3 — Assistente di Brainstorming	25
Esercizio 4 — Formatter + Brainstorming Integrati	25
Esercizio 5 — Prompt Improvement Agent	25
11 — PROSSIMA LEZIONE	26
11.1 Secondo Progetto: Canzone e Approccio Creativo	26
11.2 Spec-Driven Development	26
11.3 Claude Code	26
RIEPILOGO E CONCETTI CHIAVE	27
Concetti Chiave per Argomento	27
Progetti Claude	27
Project Knowledge e Database Vettoriale	27
Meta-Prompting	27
Cowork	27
Tabella di Riferimento Rapido	28

INTRODUZIONE

Questo manuale è il riferimento tecnico della seconda lezione del corso AI4Everyone dedicato all'utilizzo della piattaforma Claude di Anthropic. Il documento è stato generato a partire dalla trascrizione della sessione di formazione tenuta il 20 marzo 2026 e costituisce una risorsa autonoma e consultabile per tutti i partecipanti al corso.

Questa lezione segna un passaggio qualitativo importante nel percorso formativo: dall'apprendimento delle tecniche fondamentali di comunicazione con i modelli AI (affrontate nella prima sessione), si passa ora alla costruzione di ambienti di lavoro strutturati e persistenti attraverso il meccanismo dei Progetti Claude. Viene inoltre introdotto per la prima volta Cowork, lo strumento di automazione agentica della suite Claude.

Contesto e Rilevanza

Le tematiche affrontate in questa sessione sono di importanza strategica per l'utilizzo professionale di Claude. La capacità di strutturare un Progetto Claude — con le sue istruzioni, la sua knowledge base e le sue memorie — permette di passare da un utilizzo episodico e informale dello strumento a un utilizzo sistematico, orientato alla costruzione di veri e propri assistenti virtuali specializzati, calibrati sul contesto aziendale specifico.

Il filo conduttore della lezione è l'esercizio pratico del Beas Assistant: la costruzione step-by-step di un assistente virtuale dedicato al gestionale di produzione Beas Manufacturing, integrato con SAP Business One. Questo esempio reale, scelto per la sua rilevanza nel contesto Gamma, permette di toccare con mano tutte le dimensioni chiave: configurazione del progetto, gestione della project knowledge, meta-prompting per la generazione automatica delle istruzioni, e utilizzo di Cowork per task autonomi di analisi documentale.

Obiettivi di Apprendimento

Al termine di questa lezione, il partecipante sarà in grado di:

- Comprendere cos'è un Progetto Claude e in quali casi utilizzarlo
- Creare e configurare un progetto con nome, descrizione, istruzioni e file
- Distinguere i diversi livelli di memoria in Claude (globale, di progetto, di chat)
- Comprendere come funziona la project knowledge
- Applicare la tecnica del meta-prompting per generare automaticamente la configurazione di un progetto
- Utilizzare Cowork per avviare task autonomi di analisi e generazione documenti
- Progettare un assistente virtuale specializzato per un dominio tecnico specifico

1 — RIEPILOGO DELLA PRIMA LEZIONE

Prima di introdurre i nuovi argomenti, è stata condotta una breve rassegna dei concetti chiave affrontati nella lezione precedente, che costituiscono le fondamenta su cui si costruisce il lavoro di questa sessione.

1.1 Framework TCOF

Il framework TCOF (Task, Context, Output, Format) è lo standard moderno per la strutturazione dei prompt verso i modelli linguistici di grandi dimensioni. È stato presentato come lo schema più diffuso attualmente disponibile, pur riconoscendo che il panorama evolve rapidamente.

Componente	Descrizione
Task	Il compito specifico che si vuole affidare all'assistente virtuale
Context	Il contesto professionale, i vincoli tecnici e le informazioni di sfondo rilevanti
Output	Il risultato atteso: tipo di documento, formato di risposta, livello di dettaglio
Format	Formattazione specifica: markdown, elenchi, tabelle, lunghezza massima, lingua

Il framework TCOF non è solo un modo di strutturare la forma del prompt, ma definisce anche regole qualitative sul contesto: non bisogna fornire né troppo contesto né troppo poco. Quando si ha bisogno di esplorare idee, è preferibile parlare del "cosa fare" piuttosto che del "come farlo", per evitare di guidare l'AI verso soluzioni premature.



Role Prompting

Il role prompting — l'assegnazione di un ruolo professionale specifico all'assistente — fa parte del componente Context del framework TCOF. Serve a calare l'AI nel contesto operativo desiderato, migliorando la qualità e la pertinenza delle risposte.

1.2 Context Window

La context window è la "scrivania di lavoro" di Claude: l'insieme di tutte le informazioni che il modello riesce a vedere e gestire simultaneamente in una conversazione. Man mano che la chat procede e che si caricano documenti, la finestra si riempie.

Quando la context window si satura, Claude attiva un meccanismo di auto-compattazione: la parte iniziale della conversazione viene riassunta automaticamente. Questo meccanismo previene la perdita immediata di informazioni, ma la qualità del riassunto non è sempre elevata. È quindi buona norma non arrivare alla saturazione.



Saturazione della Context Window

Evitare di saturare la context window è una delle best practice fondamentali. Quando la finestra si riempie, la qualità delle risposte degrada progressivamente, anche con il meccanismo di auto-compattazione attivo. Pianificare checkpoint e riassunti espliciti è una strategia efficace per mantenere alta la qualità nel tempo.

1.3 Gestione della Memoria

Claude gestisce la memoria su livelli distinti e non sovrascriventi: ogni livello arricchisce il contesto senza cancellare quello superiore. I principali livelli sono:

Livello di Memoria	Descrizione
Preferenze Personali	Impostazioni globali configurabili nelle impostazioni utente (stile comunicativo, ruolo aziendale, ecc.)
Memoria da conversazioni precedenti	Informazioni che Claude estrapola automaticamente dalle conversazioni passate
Memoria di progetto	Istruzioni e file caricati in un progetto Claude (verrà approfondita in questa lezione)
Memoria chat (documenti condivisi)	Tutto ciò che viene caricato in una chat e che di conseguenza entra nella context window



Memoria e Scrittura

Gestire la memoria in Claude significa sempre "scrivere le informazioni da qualche parte e dare accesso a Claude a queste informazioni". Non esiste una memoria implicita o automatica: ogni informazione che si vuole rendere persistente deve essere esplicitamente registrata in uno dei livelli disponibili.

2 — FEEDBACK DEI PARTECIPANTI

All'inizio della sessione, è stato condotto un giro di tavolo per raccogliere i feedback dei partecipanti sulla prima lezione e sulle sperimentazioni effettuate nel periodo intercorso. Questo momento ha generato scambi di valore che illustrano diversi approcci pratici all'utilizzo di Claude.

2.1 Meta-Prompt come Regola nelle Preferenze

Partecipante 1 ha mostrato come ha configurato nelle proprie preferenze personali di Claude una regola con shortcut personalizzato: digitando `"/prompt"` nell'interfaccia di chat, Claude interpreta il testo successivo e genera automaticamente un prompt strutturato secondo il framework TCOF.

Questa tecnica sfrutta le preferenze personali non solo per memorizzare informazioni sul proprio ruolo, ma anche per definire comportamenti operativi ricorrenti ("regole con shortcut"). Il risultato è un assistente che non si limita a strutturare formalmente il prompt, ma lo arricchisce anche con domande contestuali — due funzioni in un solo meccanismo.



Regole e Shortcut nelle Preferenze

Le preferenze personali di Claude possono contenere non solo informazioni statiche (nome, ruolo, azienda), ma anche regole comportamentali con trigger personalizzati. Questa tecnica permette di creare micro-automazioni che velocizzano le operazioni ricorrenti senza richiedere la configurazione di un progetto dedicato.

2.2 Script Python per Automazione Documentale

Partecipante 2 ha illustrato come ha applicato le conoscenze acquisite nella prima lezione per creare script Python integrati in Visual Studio Code, finalizzati ad automatizzare la rinomina e la popolazione di file per un sistema di gestione documentale.

Questo permette di abbattere i tempi su attività ripetitive.



Automazione di Attività Ripetitive

L'utilizzo di Claude per generare script che automatizzano attività ripetitive è uno degli scenari applicativi a più alto ROI, specialmente per profili non tecnici. La chiave è identificare le attività che richiedono logica ripetibile e richiedere a Claude di generare gli script necessari, che poi possono essere eseguiti e iterati nel tempo.

2.3 Artifact come Checkpoint di Contesto

Partecipante 3 ha condiviso una tecnica di gestione della context window che ha adottato sistematicamente: creare artifact espliciti come checkpoint prima che Claude effettui la compattazione automatica. Questo approccio permette di:

- Controllare quali informazioni Claude considera più rilevanti in quel momento
- Correggere eventuali distorsioni nelle priorità (Claude potrebbe enfatizzare aspetti secondari)
- Creare un punto di partenza pulito per una nuova conversazione
- Mantenere sia l'operatore (il partecipante) sia lo strumento focalizzati sull'argomento principale

Questa tecnica si ispira alla logica scolastica: "spiegami cosa hai capito fino ad ora" è un modo per verificare la comprensione e correggere il tiro prima di procedere. Serve sia come punto di controllo sia come punto di ripartenza per non saturare le risorse.

2.4 Approccio Esplorativo e Incrementale

Partecipante 4 ha presentato un approccio alternativo al TCOF strutturato: quando l'obiettivo finale non è ancora chiaro, un prompt aperto e incrementale permette di esplorare le possibilità prima di convergere su una soluzione specifica. Questo approccio è risultato particolarmente utile per la fase di analisi iniziale di un problema tecnico.

Il Partecipante 4 ha anche introdotto una tecnica complementare: chiedere a Claude di generare il prompt ottimale per Claude stesso, partendo da un'idea grezza. Questo anticipa la tecnica del meta-prompting che verrà approfondita nella lezione.



Limiti del TCOF in Fase Esplorativa

Il framework TCOF è ottimale quando si ha già una visione chiara del risultato atteso. In fase di esplorazione o brainstorming, un prompt più aperto può essere preferibile: permette all'AI di proporre soluzioni a cui non si sarebbe pensato. L'importante è essere consapevoli del contesto in cui ci si trova e scegliere l'approccio più adeguato.

3 — PROGETTI CLAUDE

I Progetti Claude sono il nucleo tematico principale di questa lezione. Rappresentano una delle funzionalità più potenti e strategicamente rilevanti dell'ecosistema Claude, permettendo di passare da un utilizzo episodico a un utilizzo professionale e strutturato.

3.1 Cos'è un Progetto Claude

Un Progetto Claude è un ambiente di lavoro isolato e dedicato all'interno dell'interfaccia di Claude, progettato per strutturare un insieme di conversazioni focalizzate su un tema, un obiettivo o un contesto specifico. A differenza delle semplici chat, i progetti mantengono una configurazione persistente composta da istruzioni, file e memoria dinamica.

La stessa logica dei progetti è presente in altri prodotti AI sul mercato (come OpenAI con i Custom GPTs e ChatGPT Projects), ma Claude offre un'implementazione particolarmente ricca in termini di granularità della configurazione e integrazione con gli strumenti della suite.

3.2 Quando Usare un Progetto

La scelta tra una semplice chat e un progetto dipende dalla natura e dalla frequenza dell'attività:

Scenario	Strumento Consigliato
Domanda occasionale, uso una tantum	Chat normale
Ricerca rapida di informazioni	Chat normale
Tema ricorrente su cui si lavora nel tempo	Progetto
Knowledge base specializzata da interrogare più volte	Progetto
Assistente virtuale dedicato a un dominio	Progetto
Più conversazioni correlate su uno stesso argomento	Progetto
Configurazione di ruolo e comportamento persistente	Progetto



Regola Pratica

Se la domanda è "farò questa richiesta una sola volta", allora una chat normale è sufficiente. Se invece si prevede di tornare più volte su un argomento, di costruire progressivamente una knowledge base, o di avere un assistente "sempre pronto" su un certo dominio, allora il progetto è lo strumento giusto.

3.3 Struttura dell'Interfaccia Progetti

La sezione Progetti di Claude è organizzata in quattro viste:

Vista	Contenuto
Your Projects	Progetti creati personalmente e non condivisi con altri
Team	Tutti i progetti condivisi con il team (visibili a tutti i membri)
Shared with You	Progetti che un singolo utente ha condiviso solo con te (non con tutto il team)
Archived	Progetti archiviati, non attivi ma recuperabili in qualsiasi momento

I progetti possono essere ordinati per attività recente, data di creazione o data di modifica. È disponibile anche una funzione di ricerca contestuale che cerca nel titolo, nella descrizione e nel contenuto delle conversazioni.

3.4 Creazione di un Progetto

Per creare un nuovo progetto è sufficiente cliccare sul pulsante "New project" e compilare i seguenti campi:

1. Nome del progetto — un titolo descrittivo che identifichi chiaramente l'ambito
2. Descrizione — contestualizza il progetto per l'utente e fornisce contesto aggiuntivo all'AI
3. Visibilità — privata (solo per te), condivisa con il team, o condivisa con utenti specifici

Una volta creato, il progetto mostra quattro aree principali: Memory, Instructions, Files e la sezione per le Chat.

3.5 Le Quattro Aree di un Progetto

3.5.1 Memory (Memoria Autogenerata)

La sezione Memory viene popolata automaticamente da Claude dopo alcune conversazioni all'interno del progetto. Il sistema analizza il contenuto delle chat e genera note di contesto sintetiche che vengono memorizzate in quest'area. L'aggiornamento avviene tipicamente durante la notte.

Questa memoria è dinamica e personalizzata per ciascun utente: se due colleghi lavorano nello stesso progetto con approcci diversi (uno più tecnico, uno più analitico), le rispettive memorie autogenerate rifletteranno queste differenze. La memoria è visibile solo al singolo utente che l'ha generata.

3.5.2 Instructions (Istruzioni di Progetto)

Le istruzioni sono il cuore della configurazione del progetto. Corrispondono al cosiddetto System Prompt — il prompt di sistema — che definisce il ruolo, il comportamento e i requisiti permanenti dell'assistente virtuale per tutte le conversazioni all'interno del progetto.

Le istruzioni sono condivise tra tutti gli utenti del progetto (se il progetto è condiviso) e si applicano a ogni nuova chat aperta al suo interno. Includono tipicamente:

- Il ruolo professionale dell'assistente (role prompting)
- Il contesto tecnico e aziendale rilevante
- Lo stile di risposta e le preferenze di formato
- I vincoli operativi (es. "focalizzati solo sulla versione X del software")
- Le regole di comportamento (es. "se non trovi informazioni nella knowledge base, cerca sul web")



Formato delle Istruzioni

Le istruzioni di progetto vengono scritte idealmente in formato Markdown e, per motivi di efficienza nella tokenizzazione, spesso in lingua inglese. Il Markdown è il formato più "digeribile" dai modelli AI e al contempo leggibile per gli esseri umani. L'uso dell'inglese riduce il consumo di token, poiché i modelli elaborano internamente in inglese anche i prompt in altre lingue.

3.5.3 Files (Project Knowledge Base)

La sezione Files permette di caricare documenti che diventano parte della knowledge base permanente del progetto. Questi file sono accessibili in qualsiasi conversazione all'interno del progetto senza bisogno di ricaricarli ogni volta.

Formati supportati: PDF, documenti Word, file Excel, file di testo, file Markdown e altri formati documentali comuni. I file possono essere caricati direttamente dal dispositivo o importati da Google Drive.



Gestione Strategica dei File

Caricare file nella project knowledge è una decisione strategica. Vanno inclusi solo i documenti con queste caratteristiche: (1) ufficiali e validati, (2) rilevanti per tutte le conversazioni del progetto (non solo per una specifica chat), (3) stabili nel tempo (aggiornati infrequentemente). File temporanei o specifici per una singola analisi vanno invece gestiti a livello di singola conversazione.

3.5.4 Tool Access e Connettori

Nell'area di configurazione del progetto è possibile scegliere quali strumenti (tool) abilitare e con quale modalità. Questa scelta ha impatto diretto sul consumo di risorse e sulla velocità di risposta.

Modalità di Accesso ai Tool	Comportamento
Load tools when needed	I tool vengono caricati nel contesto solo quando Claude li riconosce come necessari. Risparmia risorse e mantiene il contesto più leggero (CONSIGLIATA)
Tools ready loaded	I tool sono sempre presenti nel contesto, pronti all'uso immediato. Più veloce, ma consuma più risorse di base



Linea Guida Tool Access

La modalità "Load tools when needed" è la configurazione raccomandata per la maggior parte dei casi d'uso. Consente a Claude di caricare autonomamente solo gli strumenti necessari per ciascuna richiesta, ottimizzando il consumo di token e mantenendo il contesto il più leggero possibile.

3.6 Gerarchia della Memoria in Claude

Un punto fondamentale compreso durante la sessione riguarda la struttura gerarchica della memoria in Claude. I diversi livelli non si sovrascrivono: ciascun livello eredita dal padre e aggiunge il proprio contributo.

Livello	Portata e Visibilità
Preferenze Personali	Globale — si applica a tutte le chat e tutti i progetti per l'utente specifico
Memoria da conversazioni precedenti (AI)	Globale — estratta automaticamente dalle conversazioni passate
Istruzioni e file di Progetto	Progetto — si applica a tutte le chat nel progetto; i documenti sono accessibili da qualsiasi chat nel progetto; memoria comune a tutti gli utenti del progetto
Memoria di Progetto (AI)	Progetto + Utente — autogenerata, privata per ciascun utente
Chat Corrente	Chat — file e documenti caricati nella singola conversazione

4 — WEB SEARCH VS DEEP RESEARCH

Durante la sessione è emersa una domanda importante sulla differenza tra le due funzionalità di ricerca web disponibili nell'interfaccia di Claude. Si tratta di modalità di lavoro fondamentalmente diverse, con scenari d'uso distinti.

4.1 Web Search

La Web Search è una funzionalità che permette a Claude di accedere in modo puntuale al web durante una conversazione normale. Quando viene abilitata (o anche quando viene richiesta esplicitamente dall'utente nel prompt), Claude effettua una chiamata HTTP/HTTPS verso uno o più siti web, estrapola le informazioni rilevanti e le integra nella risposta.

La Web Search si integra naturalmente nel flusso di una chat: è una singola operazione di fetch, circoscritta al momento della richiesta. Può essere attivata manualmente dall'utente o dallo stesso Claude quando riconosce la necessità di informazioni aggiornate.

4.2 Deep Research (Research)

La funzionalità Research (chiamata Deep Research da altri vendor come Perplexity e ChatGPT) è una modalità di lavoro completamente diversa. Non si tratta di un singolo accesso al web, ma di un'operazione autonoma e prolungata di ricerca, analisi e sintesi su un insieme ampio di fonti.

Funziona come affidare un compito di ricerca approfondita a un collaboratore esperto: si fornisce il tema, i requisiti e le fonti da consultare, e il sistema scandaglia autonomamente decine o centinaia di risorse web, raccoglie le informazioni rilevanti e produce un report strutturato. Un'operazione di Deep Research può richiedere dai 10 ai 15 minuti o più.

Caratteristica	Web Search	Deep Research
Portata	Singolo accesso puntuale	Centinaia di fonti in parallelo
Durata	Secondi	10–15 minuti o più
Integrazione	Seamless nella chat normale	Modalità di lavoro separata
Output	Informazione inline nella chat	Report strutturato e dettagliato
Consumo di risorse	Basso-medio	Elevato
Attivazione	Automatica o su richiesta	Solo su richiesta esplicita



Consumo di Risorse con Deep Research

La funzionalità Research consuma una quantità significativamente maggiore di risorse (token e operazioni) rispetto a una Web Search normale. Un utilizzo inconsapevole può esaurire rapidamente il piano disponibile. È raccomandata per ricerche strutturate e complesse, non per domande puntuali che possono essere soddisfatte da una singola ricerca web.

5 — EXTENDED THINKING E SELEZIONE DEL MODELLO

5.1 Extended Thinking

L'Extended Thinking è una modalità di elaborazione avanzata disponibile nei modelli Claude più recenti (in particolare Claude Sonnet 4.x). Quando abilitata, il modello effettua una catena di ragionamento più lunga ed elaborata prima di fornire la risposta finale.

Come emerso dalla discussione durante la sessione, l'Extended Thinking ha due effetti principali:

- Aumenta la qualità delle risposte attraverso un processo di reasoning più profondo e strutturato
- Estende la dimensione della context window e la lunghezza massima delle risposte che il modello può generare

Il secondo punto è meno intuitivo ma molto rilevante: senza Extended Thinking, il modello tende a produrre risposte più sintetiche. Con Extended Thinking attivo, è in grado di elaborare e restituire contenuti più estesi e dettagliati, il che è particolarmente utile per task di analisi e generazione di documenti complessi.



Abilitare Sempre Extended Thinking

La raccomandazione generale è di mantenere l'Extended Thinking sempre abilitato. L'aumento del consumo di token è compensato dal significativo miglioramento qualitativo delle risposte. Nei progetti dedicati ad attività di analisi o generazione documentale, questo fa una differenza sostanziale.

5.2 Modelli Disponibili

All'interno di un progetto Claude (e più in generale nell'interfaccia Claude), è possibile selezionare il modello da utilizzare. I modelli della famiglia Claude 4.x offrono diversi profili di capacità e consumo:

Modello	Profilo di Utilizzo
Claude Opus 4.x	Massima qualità e capacità di reasoning. Consigliato per task complessi, analisi approfondite, generazione di prompt ottimali. Consuma più risorse.
Claude Sonnet 4.x	Bilanciamento ottimale tra qualità e consumo. Adatto alla maggior parte dei task professionali. Configurazione raccomandata per l'uso quotidiano.
Claude Haiku 4.x	Veloce e leggero. Ideale per domande semplici, elaborazioni rapide. Minore consumo di risorse.



Quando Usare Opus

Per la generazione di configurazioni di progetto, prompt complessi, analisi multi-step o qualsiasi task che richieda il massimo della qualità ragionativa, vale la pena investire le risorse aggiuntive di Opus. Per le conversazioni quotidiane operative, Sonnet offre un eccellente rapporto qualità/costo.

6 — PROJECT KNOWLEDGE

Una delle spiegazioni tecnicamente più dense della sessione ha riguardato il funzionamento interno della project knowledge: come i documenti vengono memorizzati, indicizzati e recuperati dal sistema durante le conversazioni.

6.1 Come Funziona il Database Vettoriale

Quando si caricano file nella project knowledge di un progetto Claude, i documenti non vengono semplicemente archiviati come testo grezzo. Il sistema li trasforma attraverso un processo di tokenizzazione e poi li inserisce in un database vettoriale.

Un database vettoriale rappresenta ogni porzione di testo come un punto nello spazio matematico multidimensionale, in modo che concetti semanticamente simili si trovino vicini tra loro. Questo permette un tipo di ricerca fundamentally diverso dalla ricerca tradizionale:

Tipo di Ricerca	Come Funziona
Full-text search	Cerca corrispondenze esatte di parole o frasi (es. Ctrl+F in un PDF, ricerca su Google)
Ricerca semantica	Cerca per significato: trova documenti correlati anche se usano termini diversi. Es: "latte" e "uova" → trova "torta"

6.2 Estrapolazione dei Chunk

Un aspetto cruciale da comprendere è che i documenti nella project knowledge non vengono caricati interamente nella context window ad ogni conversazione. Il sistema utilizza un meccanismo di chunking — suddivisione in frammenti — e recupero semantico selettivo:

1. Quando l'utente fa una domanda, il sistema estrae i concetti chiave dalla richiesta
2. Questi concetti vengono usati per effettuare una ricerca semantica nel database vettoriale
3. Il sistema recupera solo le porzioni (chunk) dei documenti più pertinenti alla domanda
4. Solo questi chunk vengono caricati nella context window per la formulazione della risposta

Questo meccanismo ha due implicazioni fondamentali:

- Con 100 documenti caricati, la context window non si satura: vengono caricati solo i frammenti rilevanti
- Se si vuole che Claude analizzi un intero documento specifico, è necessario dirgli esplicitamente: "leggi il documento X accessibile dalla project knowledge"



Chunk e Recupero Selettivo

Claude dà l'impressione di "sapere tutto" sulla project knowledge, ma in realtà accede selettivamente alle porzioni rilevanti. Per analisi che richiedono la comprensione integrale di un documento, è necessario indicare esplicitamente il documento da leggere. Il sistema poi lo carica intero nella context window per quella specifica operazione.



Project Capacity

Una buona regola pratica è preferire meno documenti ma pertinenti e ben strutturati, piuttosto che caricare massivamente tutta la documentazione disponibile. La selettività nella scelta dei file migliora la precisione della ricerca semantica e la qualità delle risposte dell'assistente virtuale.

7 — META-PROMPTING: GENERAZIONE DEI PROMPT

Uno degli insight metodologici più potenti introdotti in questa lezione è la tecnica del meta-prompting: invece di scrivere direttamente il prompt per un task, si chiede a Claude di generare il miglior prompt possibile per svolgere quel task. Questa tecnica si può applicare a qualsiasi livello di complessità.

7.1 Principio del Meta-Prompting

Il principio è semplice: i modelli AI sono molto bravi a strutturare prompt ben formati, a rispettare framework come TCOF, e a identificare i requisiti necessari per svolgere un dato compito. Piuttosto che fare tutto il lavoro di strutturazione da soli, si può delegare questa attività all'AI stessa.

Questo crea un flusso in due fasi:

1. Fase di meta-prompting: si fornisce a Claude una descrizione informale dell'obiettivo e si chiede di generare il prompt ottimale per raggiungerlo (spesso sotto forma di artifact scaricabile)
2. Fase di esecuzione: si utilizza il prompt generato (eventualmente rivisto e approvato) per avviare il task effettivo

Il vantaggio di separare le due fasi è duplice: si ottiene un prompt qualitativamente superiore a quello che si scriverebbe direttamente, e si ha la possibilità di rivedere e approvare la "strategia" prima di avviarla.

7.2 Applicazione alla Configurazione di Progetto

Durante l'esercizio del Beas Assistant, è stato dimostrato come il meta-prompting si applica alla generazione della configurazione di un progetto Claude. Il flusso prevede due varianti:

Variante A — Prompt Diretto

Si descrive il progetto e si chiede a Claude di generare direttamente i deliverable di configurazione:

```
In questa chat sei un formatore. [Contesto tecnico aziendale]. Genera i seguenti artifact: (1) Descrizione sintetica del progetto in formato testo, (2) Project context in formato Markdown, (3) Istruzioni di progetto (system prompt) in formato Markdown.
```

Variante B — Prompt per Generare il Prompt

Si chiede a Claude di scrivere il miglior prompt per fare lo stesso lavoro — poi si esegue quel prompt:

```
Scrivi il miglior prompt per Claude in lingua italiana per richiedere la generazione dei seguenti contenuti sotto forma di artifact scaricabili: [stessa lista]. Genera questo prompt in formato Markdown sotto forma di artifact.
```

La Variante B produce generalmente un prompt più elaborato e completo. Si consiglia di confrontare i risultati delle due varianti e, per task di alta importanza, di usare il modello Opus per la fase di meta-prompting.

7.3 Qualità Riproducibile con Prompt Ben Strutturati

Un concetto fondamentale ribadito durante la lezione riguarda la natura dei modelli AI: sono sistemi non-deterministici. Ogni volta che lo stesso prompt viene eseguito, il risultato sarà leggermente diverso.

Un prompt ben strutturato non elimina questa variabilità, ma la riduce significativamente: fornisce una "strada" chiara che guida il sistema verso risultati coerenti e qualitativamente simili tra loro — anche se mai identici. La metafora del modello come "black box a strati" (descritta come una lasagna a più livelli comunicanti internamente in lingua inglese) aiuta a capire perché la struttura del prompt è così determinante.



Salvare Sempre i Prompt Efficaci

I prompt che producono risultati di alta qualità vanno salvati e versionati. Conviene adottare questa pratica sistematicamente, creando cartelle di prompt per categoria (prompt generici, prompt per Claude Code, ecc.). Questo patrimonio di prompt è un asset operativo prezioso che si costruisce nel tempo e accelera il lavoro futuro.

8 — ESERCIZIO PRATICO: IL BEAS ASSISTANT

Il cuore pratico della lezione è stato la costruzione step-by-step del Beas Assistant: un assistente virtuale dedicato al gestionale di produzione Beas Manufacturing (versione 2024.04), integrato con SAP Business One. Questo esercizio è stato scelto per la sua rilevanza diretta nel contesto Gamma e perché permette di sperimentare tutti i concetti chiave in un unico flusso di lavoro.

8.1 Problema: Accedere alla Documentazione Online di Beas

Il punto di partenza è una sfida concreta: la documentazione di Beas Manufacturing è disponibile online come Single Page Application (SPA), non scaricabile direttamente come PDF. Consultare questa documentazione ogni volta tramite web search è inefficiente e costoso in termini di risorse.

Sono state identificate tre strategie per affrontare il problema:

Strategia	Descrizione	Valutazione
Web Search puntuale	Chiedere a Claude di accedere al sito web per ogni singola domanda	Fattibile, ma costoso e non riproducibile
Materializzare la documentazione	Estrarre e caricare la documentazione come file nel progetto	Ottimale: accesso rapido, stabile e controllato (APPROCCIO SCELTO)
Claude in Chrome	Usare l'integrazione browser per navigare il sito	Potente ma in beta, con limitazioni di sicurezza, ma anche costosa

8.2 Step 1 — Creazione del Progetto

Il primo passo è la creazione del progetto dall'interfaccia Claude:

3. Cliccare su "New project"
4. Assegnare il nome: "Beas Assistant"
5. Definire la visibilità: condivisa con il team Gamma
6. Non inserire ancora istruzioni o file (si genereranno automaticamente)

8.3 Step 2 — Generazione della Configurazione tramite Meta-Prompting

Invece di scrivere manualmente la configurazione del progetto, si utilizza il meta-prompting per ottenere dall'AI stessa tutti i contenuti necessari. Il prompt di input descrive il contesto tecnico (versione Beas, database, stack SAP Business One) e richiede la generazione di tre artifact:

Artifact Richiesto	Contenuto e Utilizzo
Descrizione Sintetica	Testo breve (max 200 parole) per il campo descrizione del progetto nell'interfaccia Claude
Project Context	Documento Markdown dettagliato: obiettivi, perimetro, target d'utenza, modalità di utilizzo. Va caricato come file nella project knowledge
Istruzioni di Progetto	System prompt in Markdown per il campo Instructions del progetto: ruolo, contesto tecnico, stile di risposta adattivo (utente finale vs sviluppatore), aree di expertise

8.4 Step 3 — Configurazione del Progetto con i Deliverable

Una volta ottenuti i tre artifact da Claude, il flusso di configurazione è il seguente:

1. Aprire il progetto Beas Assistant
2. Cliccare su "Edit Details" e incollare la descrizione sintetica nel campo Description
3. Nella sezione Instructions, incollare il contenuto del system prompt (istruzioni di progetto)
4. Nella sezione Files, caricare il documento Project Context (file Markdown)
5. Verificare la configurazione iniziando una chat con "Ciao, chi sei? Come puoi aiutarmi?"

8.5 Note sulla Configurazione Generata

La configurazione autogenerata dal meta-prompting tipicamente include:

- Un ruolo professionale ben definito (es. consulente senior di SAP Business One specializzato in Beas Manufacturing)
- Il contesto tecnico completo dell'infrastruttura (versione Beas, versione SAP B1, database, stack di sviluppo)
- Uno stile di risposta adattivo: tecnico per i profili sviluppatore, funzionale per gli utenti finali
- Le aree di expertise copribili (generate dall'AI sulla base delle sue conoscenze web)
- Regole operative specifiche (es. "focalizzati sulla versione 2024.04", "se non trovi informazioni nella knowledge base, cerca sul web come seconda risorsa")



Verifica delle Istruzioni Autogenerate

Le istruzioni generate automaticamente vanno sempre verificate prima di applicarle. Alcune informazioni (come le aree di expertise) vengono generate dall'AI sulla base delle sue conoscenze generali sul software, non da documentazione certificata. È necessario controllare che siano accurate e integrare eventuali requisiti aziendali specifici.

9 — COWORK: AUTOMAZIONE AGENTICA

Rispetto ai moduli Chat (Claude Desktop base) e Code (Claude Code), Cowork si posiziona come soluzione per l'automazione di task complessi multi-step, con una modalità di lavoro fondamentalmente diversa dalla chat tradizionale.

9.1 Cos'è Cowork

Cowork è un ambiente di esecuzione agentica progettato per utenti non tecnici che vogliono sfruttare le stesse capacità di Claude Code — esecuzione autonoma di task complessi, accesso a strumenti avanzati, gestione di flussi di lavoro multi-step — senza la complessità dell'interfaccia da riga di comando.

Caratteristica	Chat	Cowork	Claude Code
Modalità di lavoro	Domanda-Risposta interattiva	Esecuzione autonoma di task	Sviluppo e analisi agentica da CLI
Interazione	Continua e bidirezionale	Minima: vai, lavora, restituisci	Controllata diff-by-diff
Target utente	Tutti	Non tecnici	Sviluppatori e power user
Motore sottostante	Modello chat	Motore agentico (come Code)	Motore agentico CLI
Stato attuale	Stabile	Preview (con limitazioni)	Stabile

9.2 Come si Avvia un Task in Cowork

Cowork è accessibile direttamente dall'interno di un progetto Claude tramite il pulsante "Start task in Cowork". Questo integra automaticamente il contesto del progetto (istruzioni, file, memory) nel task che si sta avviando.

Il flusso di lavoro tipico è:

1. Aprire il progetto Claude che contiene il contesto rilevante
2. Cliccare su "Start task in Cowork"
3. Incollare il prompt del task nel pannello di input
4. Avviare il task e monitorare la progress bar
5. Eventualmente interagire con il task in corso (fare domande, richiedere correzioni) prima che la fase rilevante sia completata
6. Ricevere l'output finale del task

9.3 Sotto-agenti e Parallelizzazione

Il motore di Cowork è basato sulla stessa architettura agentica di Claude Code. Una delle sue caratteristiche più importanti è la capacità di avviare sotto-agenti — istanze separate del modello con contesti indipendenti — per gestire operazioni parallele.

Questo ha implicazioni dirette sulla gestione delle risorse: mentre in una chat normale un task molto lungo rischia di saturare la context window, un sistema multi-agente può parallelizzare le operazioni (es. un agente per capitolo di documentazione), ri assemblando i risultati in un output finale unificato.

Strutturare il prompt in modo da richiedere esplicitamente la parallelizzazione (es. "dedica un sotto-agente a ogni capitolo, poi assembla il documento finale") è una tecnica avanzata che migliora sia la qualità sia la completezza dei risultati su task di grandi dimensioni.

9.4 Cowork per la Documentazione di Beas

Nell'esercizio pratico, Cowork è stato utilizzato per un task di analisi documentale complesso: leggere e comprendere l'intera documentazione di Beas Manufacturing accessibile online, e generare un riepilogo tecnico dettagliato in formato Markdown come artifact scaricabile.

Il prompt passato a Cowork è stato generato attraverso il meta-prompting (vedi Capitolo 7) e includeva un ordine di esecuzione critico: accesso alla documentazione → valutazione dimensionale → analisi approfondita → pre-riepilogo → generazione del report finale.

Il documento prodotto da Cowork è stato poi caricato nella project knowledge del progetto Beas Assistant, completando il ciclo: il progetto ora dispone sia della configurazione del ruolo sia della knowledge base estratta dalla documentazione ufficiale.



Strategia Ibrida per la Project Knowledge

Una strategia efficace per costruire la knowledge base di un progetto Claude prevede: (1) utilizzare Cowork per generare analisi, sintesi e riepiloghi tecnici da fonti online o documenti complessi, (2) caricare il documento generato come file nella project knowledge, (3) arricchire progressivamente la knowledge base con nuove analisi man mano che emergono nuovi requisiti.

9.5 Qualità dell'output in Cowork

Un tema rilevante emerso durante la discussione riguarda l'identificazione delle cause di un output di bassa qualità in Cowork. Le possibili cause sono molteplici e non sempre distinguibili:

Causa	Sintomo Tipico
Prompt strutturato male	L'AI non comprende il task o produce risultati fuori focus
Contesto insufficiente o eccessivo	L'AI segue una direzione sbagliata fin dall'inizio
Saturazione del contesto	Qualità che degrada progressivamente nel corso del task
Non-determinismo del modello	Risultati molto variabili tra esecuzioni diverse dello stesso prompt



Non-Determinismo e Iterazione

A causa del non-determinismo dei modelli AI, è normale che esecuzioni diverse dello stesso prompt producano risultati di qualità variabile. Prima di concludere che la causa sia la saturazione del contesto o un errore nel prompt, è utile eseguire il task più volte e confrontare i risultati. Se la variabilità è elevata, il problema è probabilmente nel prompt (insufficientemente vincolante). Se i risultati sono sistematicamente scarsi, è più probabile che il problema sia nella struttura o nel contesto.

10 — ESERCIZI ASSEGNATI

Al termine della sessione, sono stati proposti cinque esercizi da svolgere autonomamente nel periodo intercorso tra questa e la prossima lezione. Gli esercizi sono pensati per consolidare i concetti appresi e applicarli a contesti professionali reali. Non è necessario svolgere tutti gli esercizi: si consiglia di sceglierne almeno uno e approfondirlo.

Esercizio 1 — Assistente Virtuale Personale

Creare un Progetto Claude dedicato al proprio ambito tecnico specialistico, con l'obiettivo di ottenere un assistente virtuale personalizzato. Il progetto deve avere istruzioni di sistema ben definite (ruolo, contesto tecnico, stile), una knowledge base iniziale con almeno un documento di riferimento, e deve essere testato con almeno 5 conversazioni di diversa natura.



Suggerimento

Utilizzare il meta-prompting per generare automaticamente le istruzioni di progetto e il project context. Partire da un prompt che descrive il proprio ruolo aziendale e le principali esigenze informative.

Esercizio 2 — Formatter di Prompt TCOF

Creare un Progetto Claude dedicato esclusivamente alla formattazione automatica dei prompt secondo il framework TCOF, senza alterare il contenuto ma solo la forma e la struttura. Il progetto deve avere istruzioni che lo focalizzino solo sulla strutturazione formale del prompt, senza aggiungere contesti o informazioni non richieste.

Esercizio 3 — Assistente di Brainstorming

Creare un Progetto Claude dedicato al reasoning collaborativo: un assistente che ponga domande mirate per arricchire il contesto di un prompt, senza modificarne la struttura o il formato originale. L'obiettivo è aiutare l'utente a "pensare meglio" la propria richiesta prima di sottoporla all'AI.

Esercizio 4 — Formatter + Brainstorming Integrati

Combinare gli esercizi 2 e 3: creare un Progetto Claude che prima aiuta ad arricchire il contesto di un prompt attraverso domande collaborative (fase brainstorming), e poi lo struttura secondo il framework TCOF (fase formattazione). Il risultato finale deve essere un prompt completo, contestualizzato e formattato correttamente.

Esercizio 5 — Prompt Improvement Agent

Creare un Progetto Claude finalizzato al miglioramento dei prompt: dato un prompt iniziale (anche informale e non strutturato), l'assistente deve produrre un prompt di output migliorato, efficace e ben strutturato. Questo progetto differisce dall'Esercizio 2 perché non si limita a formattare il prompt esistente, ma può anche integrarlo con elementi mancanti, correggere ambiguità e ottimizzare la formulazione.

11 —PROSSIMA LEZIONE

Nella parte finale è stato anticipato il contenuto della terza lezione, che si pone come continuazione logica del percorso avviato in questa sessione. La lezione ha tre filoni principali.

11.1 Secondo Progetto: Canzone e Approccio Creativo

Verrà costruito un secondo progetto Claude con un tema completamente diverso: un assistente per la composizione di testi musicali. Questo esercizio permette di vedere come lo stesso framework dei progetti si applica in contesti non tecnici, evidenziando la versatilità dello strumento.

11.2 Spec-Driven Development

Verrà introdotta la metodologia Spec-Driven Development (SDD), che rappresenta il "filone conduttore" del corso. La metodologia sarà presentata prima su un task creativo (la composizione della canzone) e poi applicata a un contesto tecnico (sviluppo di un prototipo software).

L'approccio scelto — partire da un esempio creativo prima di passare al codice — è pedagogicamente intenzionale: vuole rendere accessibili i principi della metodologia anche ai partecipanti senza background tecnico, prima di entrare nel dettaglio dell'applicazione a scenari di sviluppo.

11.3 Claude Code

La lezione includerà anche un primo approfondimento su Claude Code, il tool a riga di comando per lo sviluppo agentivo. Verrà mostrato come Claude Code si integra nel flusso Spec-Driven Development, consentendo un controllo granulare sul processo di implementazione attraverso l'approvazione diff-by-diff delle modifiche al codice.

RIEPILOGO E CONCETTI CHIAVE

Questa sezione riassume i concetti fondamentali della seconda lezione, organizzati come riferimento rapido da consultare durante la pratica autonoma.

Concetti Chiave per Argomento

Progetti Claude

- Un Progetto Claude è un ambiente di lavoro isolato con configurazione persistente: istruzioni (system prompt), file (knowledge base) e memoria autogenerata
- Usare un progetto quando si prevede di lavorare su un tema in modo ricorrente; usare una chat semplice per domande occasionali
- Le istruzioni definiscono il comportamento dell'assistente; i file definiscono la sua knowledge base
- Istruzioni e file sono comuni a tutti gli utenti del progetto; la memoria autogenerata è privata per ciascun utente

Project Knowledge e Database Vettoriale

- I file vengono indicizzati in un database vettoriale tramite ricerca semantica
- I documenti non vengono caricati per intero nella context window: solo i chunk pertinenti vengono recuperati semanticamente
- Per leggere un intero documento, è necessario indicarlo esplicitamente nel prompt
- Caricare solo documenti stabili, ufficiali e rilevanti per tutte le conversazioni del progetto

Meta-Prompting

- Il meta-prompting è la tecnica di chiedere a Claude di generare il miglior prompt per se stesso
- Si applica in due fasi: (1) generazione del prompt ottimale, (2) esecuzione del prompt generato
- Per task complessi o di alta importanza, usare il modello Opus per la fase di meta-prompting
- I prompt ben strutturati rendono i risultati più riproducibili nonostante il non-determinismo del modello

Cowork

- Cowork è l'ambiente di esecuzione agentica della suite Claude, pensato per utenti non tecnici
- Permette di avviare task autonomi multi-step, accessibile dall'interno di un progetto tramite "Start task in Cowork"
- Il motore è agentico (come Claude Code) e può parallelizzare operazioni tramite sotto-agenti
- È attualmente in preview; consuma risorse significativamente superiori alla chat normale

Tabella di Riferimento Rapido

Scenario d'Uso	Strumento/Tecnica Consigliata
Domanda occasionale	Chat normale senza progetto
Knowledge base specializzata persistente	Progetto con file e istruzioni
Generare configurazione di un progetto	Meta-prompting (chiedere a Claude di generare le istruzioni)
Task autonomo di analisi documentale	Cowork avviato dal progetto
Ricerca puntuale sul web	Web Search nella chat
Ricerca approfondita su molte fonti	Deep Research (Research)

Fine del Manuale — Seconda Lezione